

Unité d'enseignement : Architecture N-tiers

Module : Architecture des SI II (Spring)

Enseignant(e)(s) : Chahnez SARDOUK et Sirine NAIFAR

Classe(s) : 2CINFO1 et 3C-SDE1

Documents autorisés : NON

Internet autorisé : NON

Date et Heure : .. / 12 / 2025 à 18h30

Durée : 01h30

Nbre de pages : 06

Toutes les réponses doivent être inscrites sur la feuille de réponse.

Partie 01 : QCU (07 points)

Choisir LA bonne réponse

1. Quelle annotation marque la classe principale d'une application Spring Boot ?

- A) @SpringBootApplication
- B) @EnableSpringBoot
- C) @BootApplication
- D) @MainSpring

2. Quel port utilise Spring Boot par défaut pour le serveur embarqué « TOMCAT » ?

- A) 8080
- B) 3306
- C) 8088
- D) 3308

3. Quelle annotation est utilisée pour définir une tâche planifiée dans une application Spring Boot ?

- A) @Scheduler
- B) @Scheduled

C) @Schedule

D) @TaskScheduler

4. Que signifie l'expression "0 0 12 * * ?" ?

- A) Tous les jours à minuit
- B) Tous les jours à midi
- C) Toutes les heures
- D) Toutes les 12 heures

5. Quel est l'ordre correct des niveaux de log, suivant une sévérité croissante :

- A) TRACE, DEBUG, INFO, WARN, ERROR, FATAL
- B) FATAL, INFO, DEBUG, ERROR, WARN, TRACE
- C) ERROR, DEBUG, FATAL, INFO, WARN, TRACE
- D) TRACE, ERROR, FATAL, INFO, DEBUG, WARN

6. Soit la dépendance dans pom.xml : <pre><dependency> <groupId>org.apache.httpcomponents</groupId> <artifactId>httpclient</artifactId> <version>4.1.1</version> </dependency></pre> Où se trouve le jar dans un dépôt Maven local (local repository) ? A) Dans org/apache/httpcomponents/4.1.1/httpclient B) Dans org.apache.httpcomponents.httpclient.4.1.1 C) Dans org/apache/httpcomponents/httpclient/4.1.1 D) Dans org.apache.httpclient.httpcomponents.4.1.1 7. Quelle propriété est utilisée pour changer le port à 9090 dans une application Spring Boot A) server.port = 9090 B) port = 9090 C) server = 9090 D) Il faut modifier dans les fichiers de configuration du serveur Tomcat. 8. Quelle annotation n'est pas une annotation Spring Boot ? A) @Controller B) @Service C) @Metadata D) @RestController 9. Quel module du framework Spring est utilisé pour mettre en œuvre le cross-cutting concerns dans les applications basées sur Spring ? A) ORM B) AOP C) JDBC D) Doctrine	10. Comment exécuter une requête native dans Spring Data JPA ? A) @Native B) @SQL C) @Execute D) @Query(nativeQuery = true) 11. Quelle annotation est utilisée pour spécifier une requête personnalisée dans Spring Data JPA ? A) @CustomQuery B) @JPAQuery C) @Query D) @ExecuteQuery 12. Quels types d'injection de dépendance que Spring prend-il en charge ? A) Basé sur le constructeur et les setters B) Basé sur le constructeur, les setters, et les getters C) Basé sur les setters, les getters, et les propriétés D) Toutes les réponses sont justes 13. Comment personnaliser le nom de la colonne pour un champ dans une entité JPA ? A) @Column(name="custom_name") B) @FieldName("custom_name") C) @TableColumn("custom_name") D) @DatabaseColumn("custom_name")
---	---

14. C'est quoi la cause de cette erreur ?

Caused by: com.mysql.cj.exceptions.CJCommunicationsException: Communications link failure

```
The last packet sent successfully to the server was 0 milliseconds ago. The driver has not received
at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:499) ~[na:na]
at com.mysql.cj.exceptions.ExceptionFactory.createException(ExceptionFactory.java:52) ~[mysql-c
at com.mysql.cj.exceptions.ExceptionFactory.createException(ExceptionFactory.java:95) ~[mysql-c
at com.mysql.cj.exceptions.ExceptionFactory.createException(ExceptionFactory.java:140) ~[mysql-
at com.mysql.cj.exceptions.ExceptionFactory.createCommunicationsException(ExceptionFactory.java
at com.mysql.cj.protocol.a.NativeSocketConnection.connect(NativeSocketConnection.java:79) ~[mys
at com.mysql.cj.NativeSession.connect(NativeSession.java:142) ~[mysql-connector-j-9.4.0.jar:9.4
at com.mysql.cj.jdbc.ConnectionImpl.connectOneTryOnly(ConnectionImpl.java:963) ~[mysql-connecto
at com.mysql.cj.jdbc.ConnectionImpl.createNewIO(ConnectionImpl.java:827) ~[mysql-connector-j-9.
... 48 common frames omitted
```

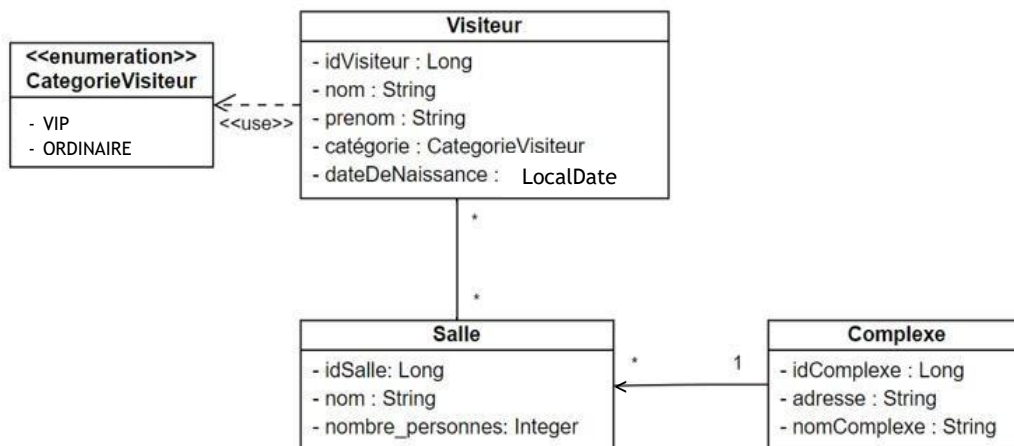
- A) La base de données MySQL n'est pas démarrée ou n'est pas accessible
- B) Le mot de passe de connexion à la base de données est incorrect
- C) La version du driver JDBC MySQL est incompatible avec la version de MySQL
- D) Le port de connexion MySQL est bloqué par un firewall

Partie 02 : Questions du cours (02 points)

- 1- Numéroté les phases suivantes dans l'ordre correct d'exécution du cycle de vie Maven (de 1 à 5) : Package, Compile, Test, Install et Deploy.
- 2- Expliquer le « PointCut » suivant: `@Before("execution(* tn.esprit.spring.service.Client.afficher*(..))")`

Partie 03 : Étude de cas : Application de gestion de salle (11 points)

On vous propose d'implémenter une application simplifiée de gestion de salle. Ci- dessous le diagramme de classes :



I- Entités et associations

1- D'après le diagramme de classes, quel est le nombre de tables générées ? Justifier la réponse.

Pour chaque entité du diagramme de classes, il faut ajouter l'ensemble des annotations JPA et les relations nécessaires afin que le schéma de base de données puisse être généré correctement.

Les points suivants doivent être respectés :

- Les identifiants des entités sont auto-générés avec la stratégie « **IDENTITY** ».
- L'association Visiteur-Salle représente une relation **bidirectionnelle**. Le visiteur est le **Child**.
- Les énumérations doivent être stockées en tant que chaîne de caractères dans la base.
- Le nom de la table générée pour chaque entité doit respecter la forme suivante : « T_(nom de l'entité) »
Exemple : Pour l'entité « Visiteur », le nom de la table générée est « T_Visiteur »
- Les mutateurs (Getters et Setters) ainsi que les constructeurs sont générés avec l'outil Lombok.
- L'injection de dépendances est réalisée avec « **les constructeurs** ».

2- Compléter les **TODOs** dans les différentes entités en prenant en considération les consignes mentionnées.

Entité « Complexe »	Entité « Salle »
<u>TODO1</u> // JPA <u>TODO2</u> // Les @s de Lombok <pre>public class Complexe { <u>TODO3</u> // JPA private Long id; private String adresse; private String nomComplexe; <u>TODO4</u> // Associations }</pre>	<u>TODO1</u> // JPA <u>TODO2</u> // Les @s de Lombok <pre>public class Salle { <u>TODO3</u> // JPA private Long id; private String nom; private int nombrePersonnes; <u>TODO5</u> // Associations }</pre>
Entité « Visiteur »	Enum « CategorieVisiteur »
<u>TODO1</u> // JPA <u>TODO2</u> // Les @s de Lombok <pre>public class Visiteur { <u>TODO3</u> // JPA private Long id; private String nom; private String prenom; private LocalDate dateDeNaissance; <u>TODO6</u> // Enumeration <u>TODO7</u> // Associations }</pre>	<pre>Public enum CategorieVisiteur { VIP, ORDIANIRE }</pre>

II- Repositories, Services et RestControllers

- 1- Compléter les **TODOs** qui permet de configurer les interfaces de la couche « Repository »

Interface « ComplexeRepository »
public interface ComplexeRepository TODO8 <Complexe, TODO9 > { TODO18 // Question 2.c }
Interface « VisiteurRepository »
public interface SalleRepository TODO8 <Salle, TODO9 > {}
Interface « SalleRepository »
public interface VisiteurRepository TODO8 <Visiteur, TODO9 > {}

- 2- Compléter le « service » ci-dessous permettant de :

- Ajouter un « Complexe » à travers la méthode « **ajouterComplexe** »
- Affecter une « Salle » à un « Complexe » à travers la méthode « **affecterSalleAComplexe** »
- Récupérer les salles d'un complexe dont son nom est passé en paramètre à travers la méthode « **afficherSallesDUnComplexe** ». Compléter le **TODO18** en utilisant « **KEYWORD** ».

TODO10 // Définir « ExamenService » comme un bean Spring qui appartient à la couche « métier »

TODO11 // Injection des dépendances

TODO12 // Outil de journalisation propre à Lombok

public Class ExamenService implements IExamenService {

TODO13 // Instance(s)

 @Override

 public void ajouterComplexe(Complexe complexe) {

TODO14 // Instruction(s) de la méthode

TODO15 // Afficher le message informatif suivant « L'ajout est effectué avec succès »

 }

 @Override

 public void affecterSalleAComplexe(Long idSalle, Long idComplexe) {

TODO16 // Instruction(s) de la méthode

TODO17 // Afficher le message avertissant suivant « L'affectation est effectuée avec succès »

 }

```

@Override
public List<Salle> afficherSallesDUnComplexe(String nomComplexe) {
    TODO19 // Instruction(s) de la méthode
}
}

```

- 3- Compléter les **TODOs** de la classe « ExamenRestController » afin d'exposer les méthodes implémentées dans la classe « ExamenService »

TODO20 // Définir « ExamenRestController » comme un bean Spring qui appartient à la couche « Exposition »

TODO11 // Injection des dépendances (la même réponse de la classe « ExamenService »)

TODO21 // Personnaliser l'accès aux méthodes de ce bean à travers le path 'api'

```

public class ExamenRestController {

    TODO22 // Instance(s)

    // Ajouter un complexe (URL du ws : http://localhost:2025/api/add)
    TODO23
    public void addComplexe( TODO24 Complexe complexe) {
        examenService.ajouterComplexe(complexe) ;
    }

    // Affecter une salle à un complexe (URL du ws : http://localhost:2025/api/assign?idC=...&idS=...)
    TODO25
    public void assignSalleToComplexe(TODO26 Long idS, TODO26 Long idC) {
        examenService.affecterSalleAComplexe (idS,idC) ;
    }

    // Afficher la liste des salles d'un complexe par son nom (URL du ws : http://localhost:2025/api/get/... )
    TODO27
    public List<Salle> getListSalleByNomComplexe(TODO28 String nomC) {
        return examenService.afficherSallesDUnComplexe(nomC);
    }
}

```

Bon courage 😊😄

